openEuler 26.09 Technical White Paper

1 Introduction

The openEuler open source community is incubated and operated by the OpenAtom Foundation.

openEuler is a digital infrastructure OS that fits into any server, cloud computing, edge computing, and embedded deployment. This secure, stable, and easy-to-use open source OS is compatible with multiple computing architectures. openEuler suits operational technology (OT) applications and enables the convergence of OT and information and communications technology (ICT).

The openEuler open source community is a portal available to global developers, with the goal of building an open, diversified, and architecture-inclusive software ecosystem for all digital infrastructure scenarios. It has a rich history of helping enterprises develop their software, hardware, and applications.

The openEuler open source community was officially established on December 31, 2019, with the original focus of innovating diversified computing architectures.

On September 30, 2020, the innovative openEuler 20.09 version was released through the collaboration efforts of multiple companies, teams, and independent developers in the openEuler community. The release of openEuler 20.09 marked a milestone not only in the growth of the openEuler community, but also in the history of open sourced software in China.

On March 31, 2021, the innovative kernel version openEuler 21.03 was released. This version is enhanced in line with Linux kernel 5.10 and also incorporates multiple new features, such as live kernel upgrade and tiered memory expansion. These features improve multi-core performance and deliver the computing power of one thousand cores.

On September 30, 2021, openEuler 21.09 was released. This premium version is designed to supercharge all scenarios, including edge and embedded devices. It enhances server and cloud computing features, and incorporates key technologies including cloud-native CPU scheduling algorithms for hybrid service deployments and KubeOS for containers.

On March 30, 2022, openEuler 22.03 LTS was released based on Linux kernel 5.10. Designed to meet all server, cloud, edge computing, and embedded workloads, openEuler 22.03 LTS is an all-scenario digital infrastructure OS that unleashes premium computing power and resource utilization.

On September 30, 2022, openEuler 22.09 was released to further enhance all-scenario innovations.

On March 30, 2023, openEuler 23.03 was released. Running on Linux kernel 6.1, it streamlines technical readiness for Linux kernel 6.x and facilitates innovations for hardware adaptation and other technologies.

On September 30, 2023, the openEuler 23.09 innovation version was released based on Linux kernel 6.4. It provides a series of brand-new features to further enhance developer and user experience.

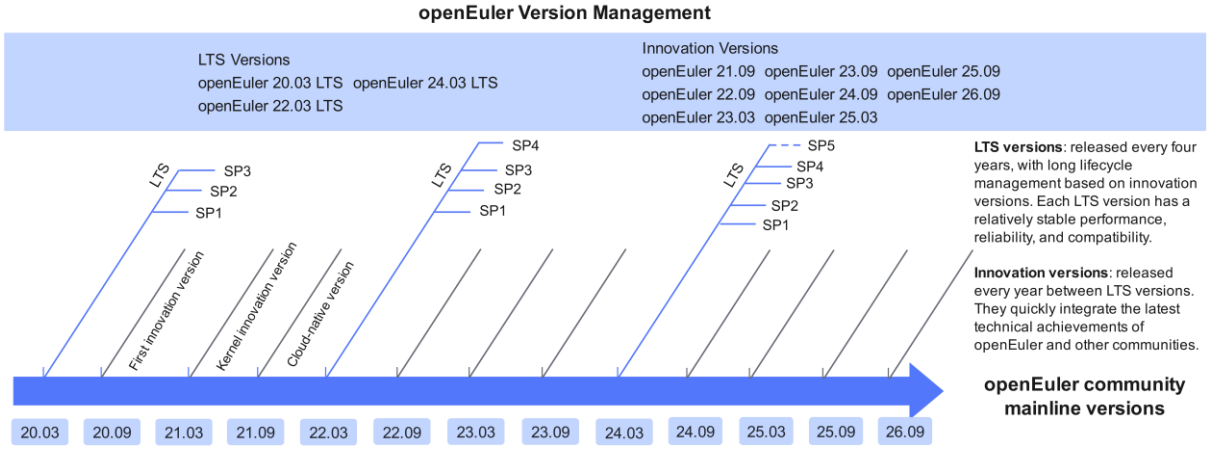
On September 30, 2024, openEuler 24.09 was released. It is an innovation version built based on Linux kernel 6.6 and brings more advanced features and functions.

On March 30, 2025, openEuler 25.03 was released. It is an innovation version designed based on Linux kernel 6.6 and is suited for server, cloud, edge, and embedded scenarios. It provides a

variety of new features and functions and brings brand-new experience to developers and users in diverse industries.

On September 30, 2025, openEuler 25.09 was released. It is an innovation version based on the Linux 6.6 kernel. It offers new features and enhanced functionality for server, cloud, AI, and embedded scenarios. Key improvements include kernel optimization, heterogeneous collaborative inference, high-density many-core computing, confidential containers, confidential VMs, multi-core and multi-instance hybrid deployment, compilers, trusted computing, and cryptographic acceleration. This latest release enhances the experience for developers and users across a wide range of fields.

On September 30, 2026, openEuler 26.09 was released as a dual-kernel innovation version based on Linux kernels 6.6 and 6.18. For AI and other scenarios, it continues to introduce new features and extend functionality based on kernel 6.6, including agent observability, SkillHub, context management, collaborative KV cache acceleration, and model loading optimization. Meanwhile, kernel 6.18 enhances the intelligent developer desktop capabilities of DevStation, delivering a new experience for developers and users and serving more domains and users.

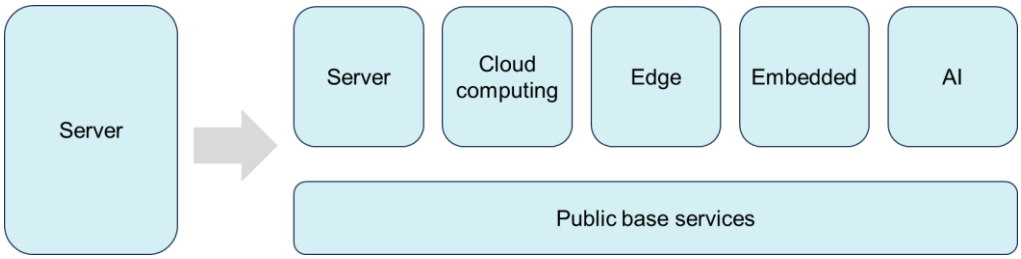


openEuler releases LTS versions to provide enhanced specifications and a secure, stable, and reliable OS for enterprise users.

openEuler is built on tried-and-tested technologies. Its innovation versions quickly integrate the latest technical achievements of openEuler and other communities. The innovative tech is first verified in the openEuler open source community as a single open source project, and then these features are added to each new release, enabling community developers to obtain the source code.

Technical capabilities are first tested in the open source community, and continuously incorporated into each openEuler release. In addition, each release is built on feedback given by community users to bridge the gap between innovation and the community, as well as improve existing technologies. openEuler is both a release platform and incubator of new technologies, working in a symbiotic relationship that drives the evolution of new versions.

Innovative Platform for All Scenarios



openEuler supports x86 and Arm processor architectures, as part of a focus to continuously improve the ecosystem of diversified computing power.

The openEuler community is home to an increasing number of special interest groups (SIGs), which are dedicated teams that help extend the OS features from server to cloud computing, edge computing, embedded, and AI scenarios. openEuler is built to be used in any scenario.

The OS is a perfect choice for ecosystem partners, users, and developers who plan to enhance scenario-specific capabilities. By creating a unified OS that supports multiple devices, openEuler hopes to enable a single application development for all scenarios.

Open and Transparent Software Supply Chain

The process of building an open source OS relies on supply chain aggregation and optimization. To ensure reliable open source software or a large-scale commercial OS, openEuler comprises a complete lifecycle management that covers building, verification, and distribution. The brand regularly reviews its software dependencies based on user scenarios, organizes the upstream community addresses of all the software packages, and verifies its source code by comparing it to that of the upstream communities. The build, runtime dependencies, and upstream communities of the open source software form a closed loop, realizing a complete, transparent software supply chain management.

2 Platform Architecture

System Framework

openEuler is an innovative open source OS platform built on kernel innovations and a solid cloud base to cover all scenarios. It is built on the latest trends of interconnect buses, storage media, and AI, and offers a distributed acceleration engine and base services. It provides competitive advantages in edge, embedded, and AI scenarios, and is the first step to building an all-scenario digital infrastructure OS.

openEuler 26.09 runs on Linux kernels 6.6 and 6.18 and provides POSIX-compliant APIs and OS releases for server, cloud native, edge, embedded, and AI environments. It is a solid foundation for intelligent collaboration across hybrid and heterogeneous deployments. openEuler 26.09 is equipped with a distributed soft bus and a K3s edge-cloud collaboration framework, further enhancing collaboration across digital infrastructure and laying the foundation for an interconnected world.

In the future, the openEuler open source community will continue to innovate, aiming to promote the ecosystem and consolidate the digital infrastructure.

All scenarios:

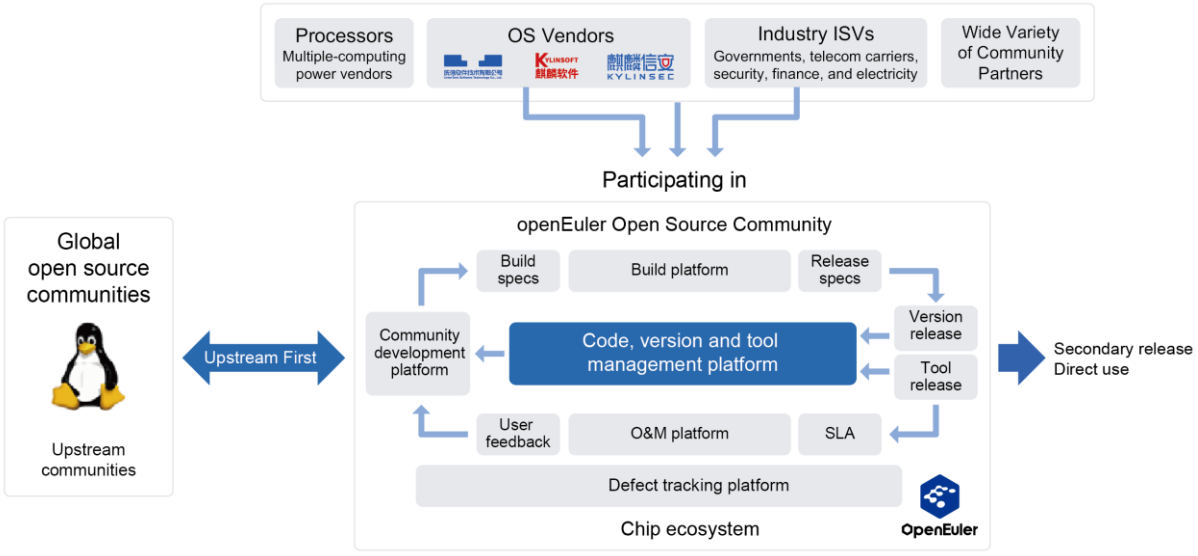
- **AI-native OS**: It supports multi-level KV caching and aggregated data transfer. Semantics-aware KV cache scheduling for agents reduces costs and improves efficiency in inference scenarios. Agent observability ensures reliable agent operation through non-intrusive, low-overhead monitoring across the entire execution chain.

Flourishing community ecosystem:

- **DevStation**: An intelligent LiveCD desktop developer workstation based on openEuler, designed to provide a ready-to-use, efficient, and secure development environment that streamlines the entire workflow from deployment, coding, compilation, and building to release. By integrating a self-developed installation tool, it significantly reduces the size of community images, enabling fast downloads, installation, and trial use. The newly added MCP-based AI engine enables rapid invocation of community toolchains, significantly improving efficiency from infrastructure setup to application development. An enhanced environment for deploying AI tools further improves the developer experience when integrating related agent applications in openEuler environments.
- **EPKG**: A lightweight, portable, cross-platform package manager for Linux, macOS, and Windows. It supports creating isolated environments and installing and using packages from popular ecosystems and distributions, including RPM, DEB, Alpine, Arch, Conda, Homebrew, and MSYS2, without requiring root privileges. Each environment is associated with a channel, such as a specific version of Debian, Fedora, or Alpine. Multiple environments can be registered, and their binaries combined in *PATH*, allowing software from different distributions to be used together in a single shell.

Platform Framework

The openEuler open source community partners with upstream and downstream communities to advance the evolution of openEuler versions.



Hardware Support

The openEuler open source community works with multiple vendors to build a vibrant southbound ecosystem. With participation of major chip vendors including Intel and AMD, all openEuler versions support x86 and Arm architectures, and a wide range of CPU chips, such as Zhaoxin KaiXian and KaiSheng, Intel Sierra Forest and Granite Rapids, and AMD EPYC 4/5. openEuler

can run on servers from multiple hardware vendors and is compatible with NIC, RAID, Fibre Channel, GPU & AI, DPU, SSD, and security cards.

openEuler supports the following CPU architectures:

Hardware Type	x86_64	AArch64
CPU	Intel, AMD, Hygon, Zhaoxin	Kunpeng, Phytium

3 Operating Environments

Servers

To install openEuler on a physical machine, check that the physical machine meets the compatibility and minimum hardware requirements.

For a full list, visit https://openeuler.org/en/compatibility/.

Item	**Configuration Requirement**
Architecture	AArch64, x86_64
Memory	≥ 4 GB
Drive	≥ 20 GB

VMs

Verify VM compatibility when installing openEuler.

Hosts running on openEuler 26.09 support the following software packages:

- libvirt-9.10.0-37.oe2609
- libvirt-client-9.10.10-37.oe2609
- libvirt-daemon-9.10.10-37.oe2609
- qemu-8.2.0-83.oe2609
- qemu-img-8.2.0-83.oe2609

openEuler 26.09 is compatible with the following guest OSs for VMs:

Guest OS	**Architecture**
CentOS 6	x86_64
CentOS 7	AArch64
CentOS 7	x86_64
CentOS 8	AArch64
CentOS 8	x86_64
Windows Server 2016	x86_64

Guest OS	Architecture
Windows Server 2019	x86_64

Item	Configuration Requirement
Architecture	AArch64, x86_64
CPU	≥ 2 CPUs
Memory	≥ 4 GB
Drive	≥ 20 GB

Edge Devices

To install openEuler on an edge device, check that the edge device meets the compatibility and minimum hardware requirements.

Item	Configuration Requirement
Architecture	AArch64, x86_64
Memory	≥ 4 GB
Drive	≥ 20 GB

Embedded

To install openEuler on an embedded device, check that the hardware meets the compatibility and minimum hardware requirements.

Item	Configuration Requirement
Architecture	AArch64, AArch32, x86_64
Memory	≥ 512 MB
Drive	≥ 256 MB

4 AI Innovations

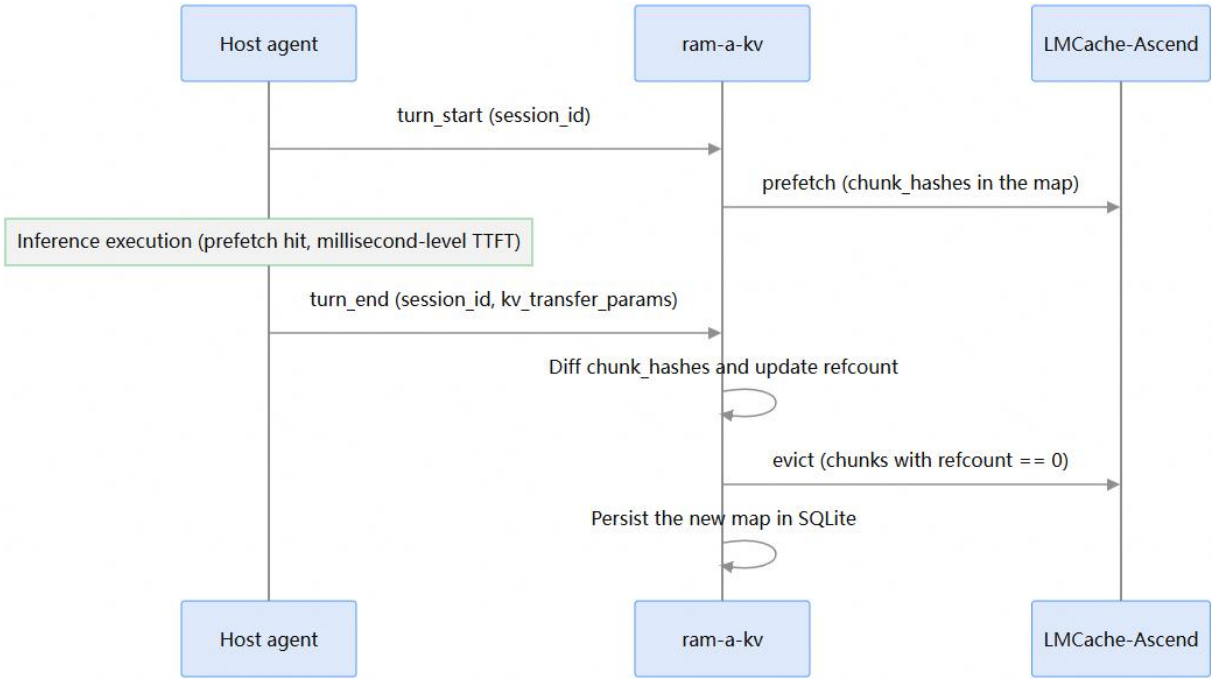
In the AI era, OSs must continue to evolve to meet the needs of AI. On the one hand, AI should be integrated throughout the OS lifecycle, from development and deployment to O&M, making the OS smarter. On the other hand, as the runtime environment for AI, the OS must provide coordinated management of heterogeneous resources for AI and agents. This enables CPUs and AI accelerators to deliver their full potential, improving business responsiveness and reducing latency. openEuler is already among the first to support mainstream AI processors, including NVIDIA and Ascend, making it a preferred choice for enabling diverse computing power.

Semantic-Aware KV Cache Coordination and Scheduling for Agents

Feature Description

In multi-turn AI agent inference, each inference turn depends on intermediate results from previous turns. These results are stored as KV cache in accelerator memory or an associated KV cache repository. However, existing KV cache systems essentially act as passive repository managers. They manage the cache using only the least recently used (LRU) policy, without understanding the semantics of agent contexts. Under multi-tenant workloads, hot cache chunks that are still needed by active sessions are often mistakenly evicted, causing time to first token (TTFT) to surge from milliseconds to seconds.

ram-a-kv acts as a semantic intelligence layer between the inference stack and the KV cache repository. It maps agent contexts to KV cache chunks and predicts their reuse patterns. Before the next inference turn, it proactively orchestrates prefetching and eviction, while using global reference counting to protect chunks shared across sessions. This brings TTFT back down to the millisecond range.



ram-a-kv runs as a standalone daemon. The host agent sends events over HTTP, and the daemon orchestrates the underlying KV cache backend (LMCache-Ascend) accordingly. The workflow is shown in the figure above.

The core components include:

- **HTTP event entry point**: The only routing endpoint, POST /event, responsible for authentication and field validation.
- **Event registry**: Dispatches nine types of events to their respective handlers.
- **Session state management**: Maintains a chunk map for each session and a global reference count, protected by an in-memory mutex.
- **SQLite persistence layer**: Stores session maps, turn_count, and the pins table.

Event-driven Lifecycle Management

Nine events are defined to cover the full lifecycle of inference: **turn_start / turn_end** (start and end of each inference turn), **snapshot_restore / session_suspend / session_close** (snapshot restoration, session suspension, and session closure), **session_fork / session_fork_end** (session forking), **session_map** (state query), and **health** (health check). The host agent only needs to report events at key points. The daemon automatically handles prefetching, eviction, and state updates.

Reference Count Safety

The core safety invariant is that a chunk can be evicted only when its global reference count reaches zero. Chunks shared across sessions are never prematurely deleted by an individual session. **turn_end** updates reference counts by computing the difference between the old and new chunk_hashes. **session_fork** and **session_fork_end** symmetrically increment and decrement reference counts to protect parent and child sessions. On startup, the daemon traverses all session maps to rebuild the refcount table, ensuring that shared chunks remain protected after a restart.

Persistence and Restart Recovery

All session maps are persisted to SQLite in real time. When the daemon restarts, it loads all sessions into memory and rebuilds the refcount table. **session_suspend** preserves the corresponding SQLite rows for subsequent recovery through **snapshot_restore**, allowing session state to resume seamlessly after a restart or suspension.

Application Scenarios

- **Scenario 1: Multi-turn conversational agent (typical use case)**
  - **Problem**: In multi-turn conversations between users and AI assistants, each inference turn depends on the context from previous turns. As the conversation grows longer, the total KV cache size exceeds accelerator memory capacity. The passive LRU policy evicts frequently accessed chunks to slower storage, causing TTFT to spike.
  - **Solution**: The host agent reports turn_start and turn_end events. Based on these events, ram-a-kv prefetches the chunks required for the current turn and evicts only chunks that are no longer referenced.
  - **Benefits**: TTFT remains in the millisecond range, even for long conversations.
- **Scenario 2: Session suspension and resumption**
  - **Problem**: Users may pause a conversation, for example, when switching tabs or resuming a chat the next day. The KV cache may be reclaimed during the pause. Without restoring the cached state, the system must recompute all historical tokens when the conversation resumes, incurring substantial overhead.
  - **Solution**: When a session is suspended, the host agent reports session_suspend. The daemon persists the pin information and releases in-memory references while retaining the SQLite rows. When the session resumes, the host agent reports snapshot_restore, prompting the daemon to load the map from SQLite and prefetch the required chunks, allowing the session to continue from its previous state.
  - **Benefits**: Users can resume conversations seamlessly, while underlying resources are released as needed rather than occupying accelerator memory indefinitely.
- **Scenario 3: Session forking**
  - **Problem**: A new branch is created from an existing session. As the parent session progresses, the child session still needs to reference chunks shared from earlier in the conversation.

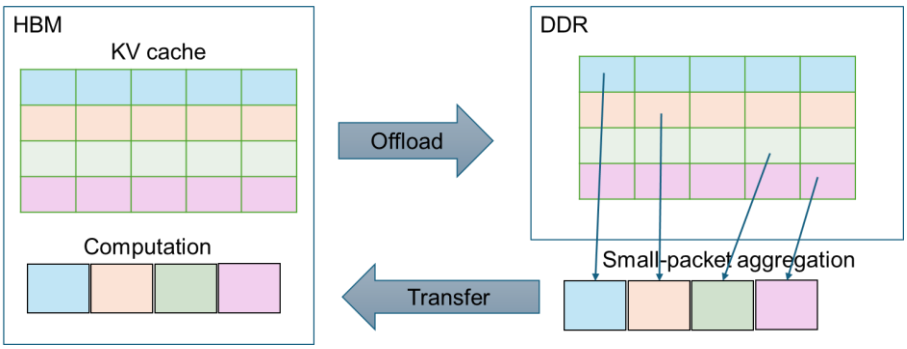
  - **Solution**: When a session is forked, the host agent reports session_fork, and the daemon increments the reference count of the current chunks by one. When the child session no longer shares these chunks, the host agent reports session_fork_end, and the daemon decrements their reference counts by one.
  - **Benefits**: Inference in one branch does not disrupt another. Shared chunks remain protected from eviction until their last reference is released.
- **Scenario 4: Multi-tenant inference services**
  - **Problem**: An inference service handles multiple users and sessions, while the KV cache backend has limited capacity. Cache resources must be scheduled fairly and efficiently across tenants.
  - **Solution**: ram-a-kv maintains a global reference count table shared across sessions. A chunk is never evicted as long as it is still referenced, preventing one tenant from evicting chunks that another tenant is still using.
  - **Benefits**: Semantically safe isolation across tenants and maximum backend capacity utilization, with unreferenced chunks evicted to free up space.

KV Cache Aggregated Transfer

Feature Description

As large language models (LLMs) scale from tens of billions to hundreds of billions or even trillions of parameters, the primary bottleneck for mainstream inference services has shifted from insufficient computing power to limited accelerator memory. For example, the DSA 3.2 model is expected to require more than 30 GB of HBM to store the KV cache for an input sequence of millions of tokens. However, thanks to the model's innovative design, only the top-K elements need to participate in attention computation during each inference step. This creates the conditions for coordinated use of DDR and HBM.

This feature implements coordinated use of DDR and HBM within a node for KV cache management during the decode phase, targeting sparse attention workloads. After prefill, the full KV cache is offloaded to DDR, while HBM retains only the indexer KV cache and a portion of the KV cache to maintain a certain cache hit rate. During layer attention computation, the required data is loaded on demand, with data transfers aggregated by the MemFabric component. This approach alleviates request blocking caused by insufficient HBM capacity under high concurrency, reduces latency, supports larger batch sizes, and lowers time per output token (TPOT).



Key optimizations based on vLLM:

- **H2D transfer optimization**: Improves effective data transfer bandwidth.

- **KV cache hit rate optimization**: Maintains a cache hit rate above 90%, reducing the amount of data transferred per operation.

Application Scenarios

In LLM inference acceleration scenarios with prefill-decode disaggregation or prefill-decode fusion, this feature optimizes the decode phase by leveraging host memory as an extension of HBM capacity under the same hardware configuration. It builds on vLLM's offloading capabilities to reduce request queuing under high concurrency, support larger batch sizes, and increase the number of tokens generated per second.

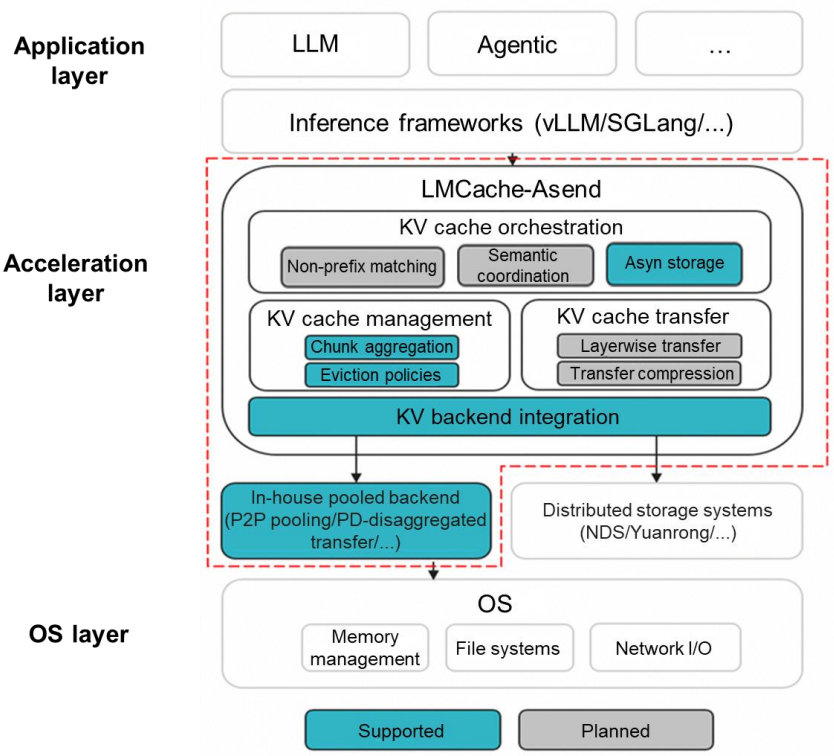
Multi-Level KV Cache

Feature Description

The interaction patterns of LLMs have changed dramatically, from single-turn question answering to multi-turn conversations and now to agent workflows. A typical agent task, such as code generation, scientific research and analysis, or office automation, may involve dozens or even hundreds of tool calls, reasoning iterations, and context continuations. This poses new challenges for KV cache management backends.

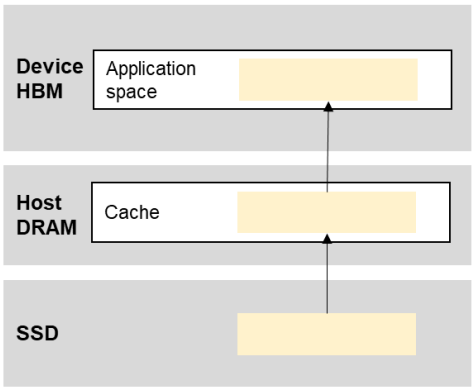
- **Dramatic growth in per-request KV cache size**: Agents must maintain complete conversation histories, retrieved context, and intermediate reasoning results. Context lengths can easily reach 128K to 1M tokens, causing the KV cache size per request to grow from megabytes to gigabytes. Since HBM cannot accommodate the entire cache, it must be offloaded across multiple storage tiers, including DDR, SSDs, and pooled memory across supernodes. The backend must therefore do more than simply move data. It must manage the KV cache lifecycle across multiple storage tiers, making data transfer and scheduling a bottleneck in their own right.
- **Sharp increase in prefix reuse**: Multiple agent instances often share the same system prompts, role definitions, and knowledge base prefixes, resulting in high KV cache hit rates. The backend must locate the longest matching cached prefix within milliseconds, reusing cached data to skip redundant prefill computation.

To address these challenges, LMCache-Ascend, a heterogeneous, integrated multi-level KV cache pool, enhances its proprietary KV cache pooling and prefill-decode transfer capabilities while providing a storage backend interface compatible with high-performance distributed storage backends such as Mooncake and YuanRong DataSystem. This further improves end-to-end performance.



As LLMs grow in size, HBM and DDR can no longer accommodate all the KV cache generated by daily inference requests, necessitating offloading to SSDs. For example, a DeepSeek cluster generates 776 billion tokens per day, corresponding to approximately 14.5 PB of KV cache data. However, existing mainstream SSD offloading solutions have performance limitations.

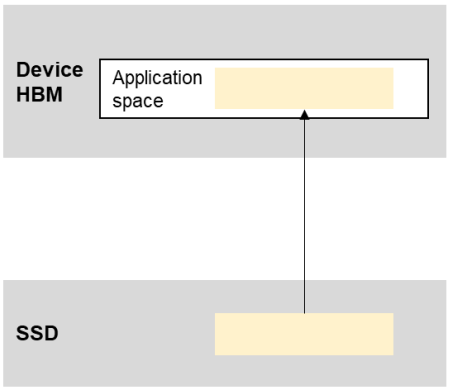
Architecture 1: Host-based I/O



Limitations

- Long I/O paths and redundant data copies on cache misses
- Unable to store the entire dataset in DRAM or on a single node's SSD

Architecture 2: GPU Direct IO

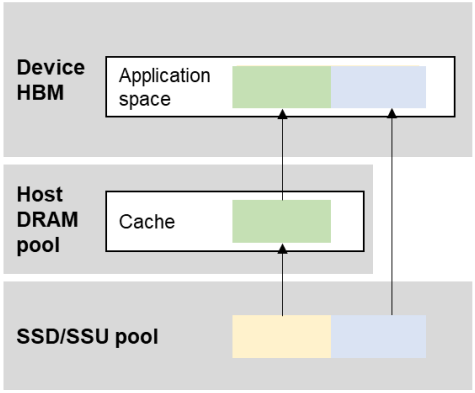


Limitations

- SSD-to-HBM transfers are still slower than DRAM-to-HBM transfers, limiting cache utilization and delivering lower performance than the DRAM-hit scenario in Architecture 1.
- Cross-node transfers incur additional protocol conversion overhead and do not provide true direct access.

To improve overall KV cache performance, hybrid I/O is implemented to leverage storage device bandwidth through direct I/O and make full use of host DDR bandwidth through caching. Pooling further aggregates read and write bandwidth.

Architecture 3 (ours): Hybrid I/O



Advantages

- Multi-channel I/O makes full use of existing hardware and enables optimal routing.
- Global pooling increases the KV cache hit rate.

1. LMCache multi-level caching

The overall architecture of LMCache is designed around one core objective: to move the KV cache beyond the inference engine's process boundary and into an independent caching layer that can be shared across processes and nodes. This is achieved through a layered architecture consisting of a connector layer, an engine layer, and a storage layer, combined with a hybrid deployment model that uses both in-process components and standalone service processes.

Layer	Core Responsibility	Key Component
Connector	Acts as a vLLM plugin to intercept KV cache operations and translate between vLLM's physical block view and LMCache's logical token view, while coordinating cache operations.	LMCacheConnector, RequestTracker, ReqMeta
Engine	Serves as the core controller, coordinating the cache lifecycle, I/O operations, and execution of tiered storage policies.	LMCacheEngine, TokenDatabase, StorageManager
Storage	Manages multi-level storage backends, including memory, P2P, disk, and remote storage, and implements data write and retrieval policies.	LocalCPUBackend, P2PBackend, LocalDiskBackend, RemoteBackend

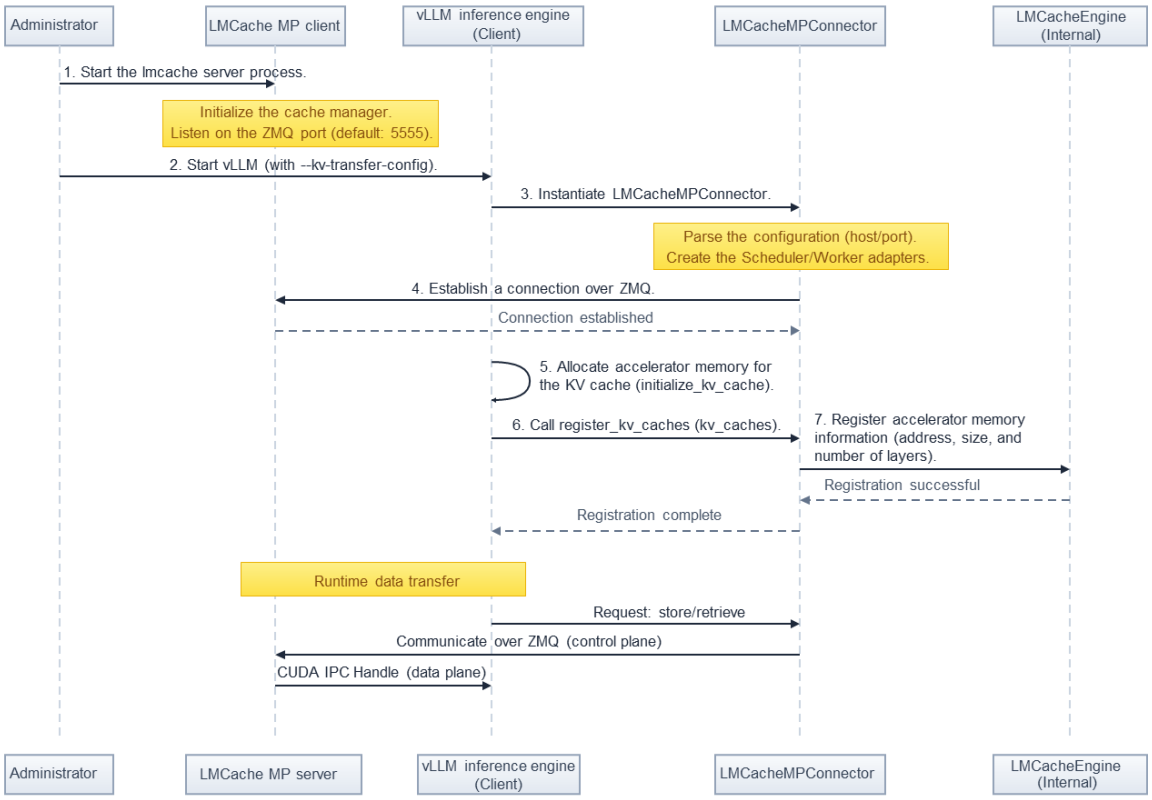
Integrating LMCache with the vLLM/SGL inference engine:

- **Server initialization and listening**: After the LMCache server process starts, it initializes the internal cache manager and begins listening for client connection requests on the specified ZMQ port.

  vllm serve <model> \

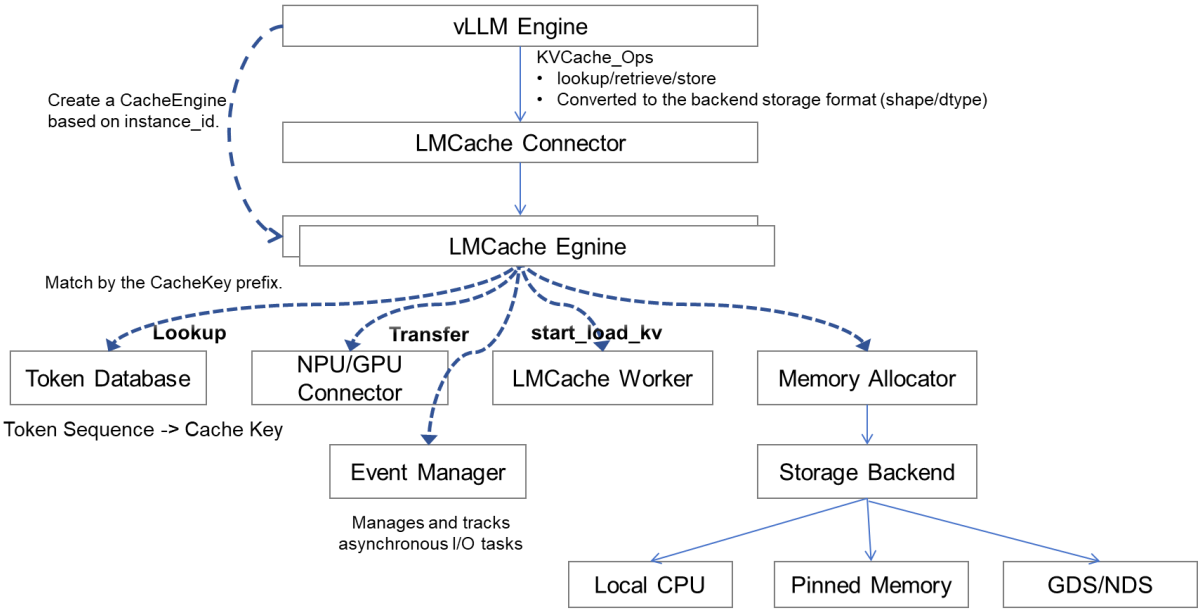
  --kv-transfer-config \

  '{"kv_connector": "LMCacheMPConnector", "kv_role": "kv_both"}'



- **vLLM connection and registration**: During initialization, the vLLM process instantiates LMCacheMPConnector based on the configuration specified in **--kv-transfer-config**. The connector establishes a connection to the configured LMCache server address using the ZMQ protocol. Once connected, vLLM calls the register_kv_caches API to register information about the per-layer KV cache accelerator memory allocated within the process, including memory addresses, sizes, and the number of layers, with the LMCache server.
- **Runtime data transfer**: After initialization, vLLM can transfer data to and from the server through LMCacheMPConnector.
- **Storage**: After vLLM generates the KV cache, the connector performs a gathering operation to aggregate KV blocks scattered across multiple GPU memory pages into a contiguous buffer.
- **Efficient transfer**: To efficiently transfer data across process boundaries, MP mode prioritizes the use of inter-process communication (IPC). Instead of copying the entire dataset, the worker process exports the accelerator memory as a lightweight IPC handle and sends it to the LMCache server. After importing the handle, the server can directly access the worker process's NPU or GPU memory, enabling highly efficient, near-zero-copy data transfer.

Overall technical components:



LMCache integrates with the inference engine through the KV connector interface:

Operation Stage	Interface Name	Core Responsibility
Initialization and registration	register_kv_caches	Registers the KV cache accelerator memory tensors allocated by vLLM with the connector during worker initialization, enabling subsequent management.
Lookup and decision-making	get_num_new_matched_tokens	Called by the scheduler when scheduling a request to determine how many additional tokens, beyond those already computed locally, can be loaded from the external KV cache.
Preloading and state synchronization	update_state_after_alloc	Called after vLLM allocates physical blocks for a request. Synchronizes the physical block information and the number of external tokens to be loaded with the connector, preparing for subsequent loading.
Initiating asynchronous loading	start_load_kv	Called by the worker before the model's forward pass begins to initiate an asynchronous task for loading the external KV cache.
Waiting for layer-by-layer loading	wait_for_layer_load	Called before the attention computation of each model layer. Blocks until the KV cache data required by that layer is ready, enabling pipelined loading.
Asynchronous layer-by-layer	save_kv_layer	Called after each model layer finishes computation to asynchronously save the KV cache generated by that layer

Operation Stage	Interface Name	Core Responsibility
saving		to external storage.
Waiting for saving to complete	wait_for_save	Called at the end of the model's forward pass. Blocks until all asynchronous save operations for all layers are complete, ensuring data integrity.
Lifecycle finalization	request_finished	Called when request processing is complete and its KV cache blocks are about to be released. If **True** is returned, the connector takes over lifecycle management of the blocks asynchronously.
Querying asynchronous completion status	get_finished	Allows the scheduler to query which requests have completed their asynchronous save or load operations so that the associated resources can be safely released.

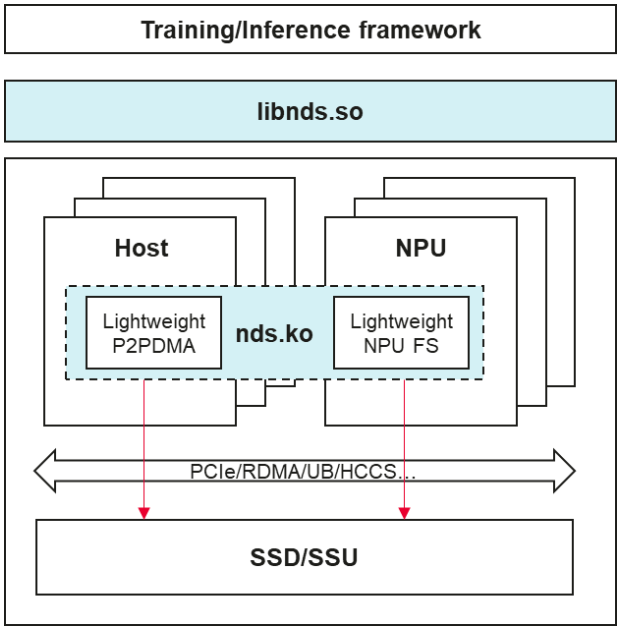
2. NPU Direct Storage

Currently, file I/O for NPUs is routed through the CPU, which introduces two issues:

- **Additional data copies**: Data must be copied along the path from SSD to DDR to HBM.
- **Bus bandwidth bottleneck**: The bus bandwidth between the CPU and the NPU and storage devices may be lower than the bus bandwidth between the NPU and storage devices.

Direct data-path access can address these two key issues, improving performance while reducing host CPU utilization and DDR bandwidth consumption. NDS has the following design goals:

- **Non-intrusive modifications**: Avoid intrusive modifications to the kernel storage stack and drivers.
- **Low overhead**: Reduce or offload control-plane overhead and minimize memory overhead.
- **High concurrency and performance**: Achieve bandwidth at least 10% higher than GPUDirect Storage (GDS).



NDS is an end-to-end framework that enables direct data movement between NVMe SSDs and NPU HBM, eliminating unnecessary host memory involvement. It consists of the following components:

- **Kernel module**: Avoids intrusive kernel modifications and the shadow buffer memory overhead associated with GDS, enabling the data path to bypass the CPU. It implements a character device that integrates the file system, block layer, and NVMe driver to construct NVMe SGL commands directly and supports initiating and querying I/O operations.
- **Integration with the LMCache ecosystem**: Implements an LMCache pooled storage backend to integrate NDS with LMCache, transparently replacing the traditional disk loading path. It includes pooled KV cache file management, LRU, promotion, and other functions.

NDS provides both C and Python APIs. The following table lists the C APIs as an example.

API Category	C API
Initialization	int nds_init(struct nds_init_param *param) int nds_exit(void)
File system registration	int nds_register_fs(const struct nds_fs_desc *desc) int nds_unregister_fs(const struct nds_fs_desc *desc)
HBM registration	int nds_register_mem(void *addr, uint64_t size, int flags) int nds_unregister_mem(void *addr, uint64_t size, int flags)
HBM read/write: I/O context management	int nds_io_new_ctx(const struct nds_io_ctx_param *param, struct nds_io_ctx **ctx) int nds_io_destroy_ctx(struct nds_io_ctx *ctx)
HBM read/write: submitting I/O requests	int nds_io_submit(struct nds_io_ctx *ctx, int nr, const struct nds_io_cb *iocb)
HBM read/write: waiting for I/O requests to complete	int nds_io_getevents(struct nds_io_ctx *ctx, int min_nr, int nr, struct nds_io_event *events, struct timespec *timeout)

Application Scenarios

The core value of the LMCache pooled system lies in transforming the KV cache from temporary data within the inference engine into a persistent resource that can be reused across requests and instances. Its applications primarily focus on reducing redundant computation. The following are some of the most typical scenarios.

- Multi-turn conversations and long sessions

  In multi-turn conversations, each turn includes prior context history, for which the corresponding KV cache remains strictly identical. Without LMCache, the entire history must be prefilled again for every request, resulting in substantial computational overhead. LMCache caches the KV cache generated by previous turns so that it can be reused directly in subsequent turns. Measured results show that, in multi-turn conversation scenarios, LMCache reduces TTFT by a factor of 4.4 while increasing the number of supported active users from 2 to 8. For long-session scenarios, measured TTFT speedup can reach 37 times.

- KV cache transfer in prefill-decode disaggregation

  In a prefill-decode disaggregated deployment, prefill nodes process inputs, while decode nodes generate outputs. The KV cache computed by a prefill node must be efficiently transferred to the decode node.

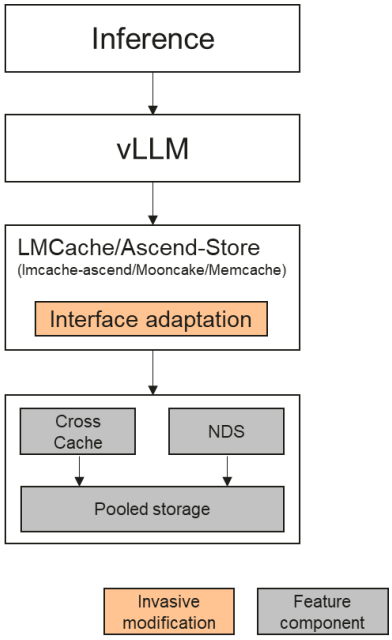
  LMCache serves as a real-time transfer layer in this architecture. Through its gather/scatter mechanism, it aggregates KV blocks scattered across vLLM's paged memory into larger chunks, then transfers them to decode nodes over RDMA channels provided by NIXL or Mooncake. Meanwhile, LMCache's MultiConnector capability allows a prefill node to transfer KV cache to decode nodes while simultaneously offloading it to a local shared cache, enabling the same computation to serve multiple destinations.

- Cross-instance sharing and load balancing

  In a standard vLLM deployment, multiple inference instances typically maintain independent KV caches, creating cache silos. If the same request is routed to different instances, the cache on one instance cannot be reused by another, even when their content is identical.

  LMCache addresses this issue through its peer-to-peer and multiprocess modes. Multiple vLLM instances can share a KV cache pool in CPU memory through LMCache or retrieve KV cache directly from peer nodes over NIXL/RDMA.

The primary use case for NPU Direct Storage is currently accelerating KV cache loading for inference services. The following figure shows the overall architecture. At the data plane, the integration primarily involves LMCache/LMCache-Ascend and NDS; Ascend-Store has not yet been integrated. The integration requires intrusive modifications to LMCache and LMCache-Ascend, without modifying other third-party components.



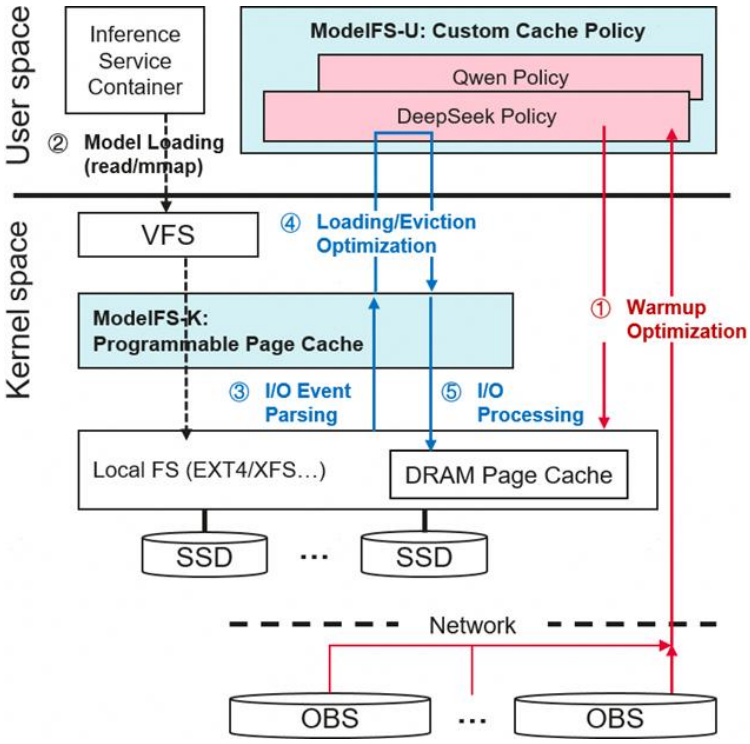
Achieved benefits and roadmap:

- **Service rollout**: MiniMax M2.5/2.7, GLM-5, and Qwen 3.5 are now available for coding services, reducing TTFT by 25%.
- **Integration completed**: UBSIO and NDS have been integrated with each other.

ModelFS User-Space Model Loading Optimization

Feature Description

ModelFS addresses the model loading bottleneck during the startup phase of large language model (LLM) inference. While existing approaches often sacrifice compatibility for performance, real-world industrial deployment demands seamless integration with existing ecosystems. ModelFS bridges this gap by achieving state-of-the-art loading performance through optimized file system cache policies, without compromising compatibility. Specifically, it introduces a non-intrusive, flexible, and lightweight programmable page cache framework in the kernel, enabling users to customize file system page cache policies. Building on this framework, we provide a reference implementation optimized to accelerate model loading.



- ModelFS-K

  The kernel module of ModelFS provides a programmable framework for the file system page cache. Its core design is a stacked file system that can be mounted on top of an existing file system. ModelFS-K redirects the logic of the underlying file system to user-space **prefetch()** and **evict()** functions via user-space procedure calls (UPCs).

- ModelFS-U

  The user-space module of ModelFS provides the runtime environment for cache policies. It offers a VFS-like programming framework in user space, allowing model implementers and I/O optimizers to customize cache policies based on the I/O characteristics of model loading. Users need to implement the **init()**, **exit()**, **prefetch()**, and **evict()** functions. ModelFS-U registers these functions with ModelFS-K and is responsible for parsing I/O events as well as performing asynchronous data prefetching and eviction at runtime.

Application Scenarios

ModelFS can accelerate I/O operations across multiple types of storage backends, including local file systems, distributed file systems, and object storage service (OBS). The following are two typical use cases.

- Remote OBS weight prefetching

  When model weights are stored in remote OBS, the inference system typically prefetches hotspot weights to local storage in advance due to the low bandwidth of OBS. ModelFS can accelerate weight prefetching by implementing a cache policy that aggregates I/Os across multiple OBS buckets.

- Local SSD weight loading acceleration

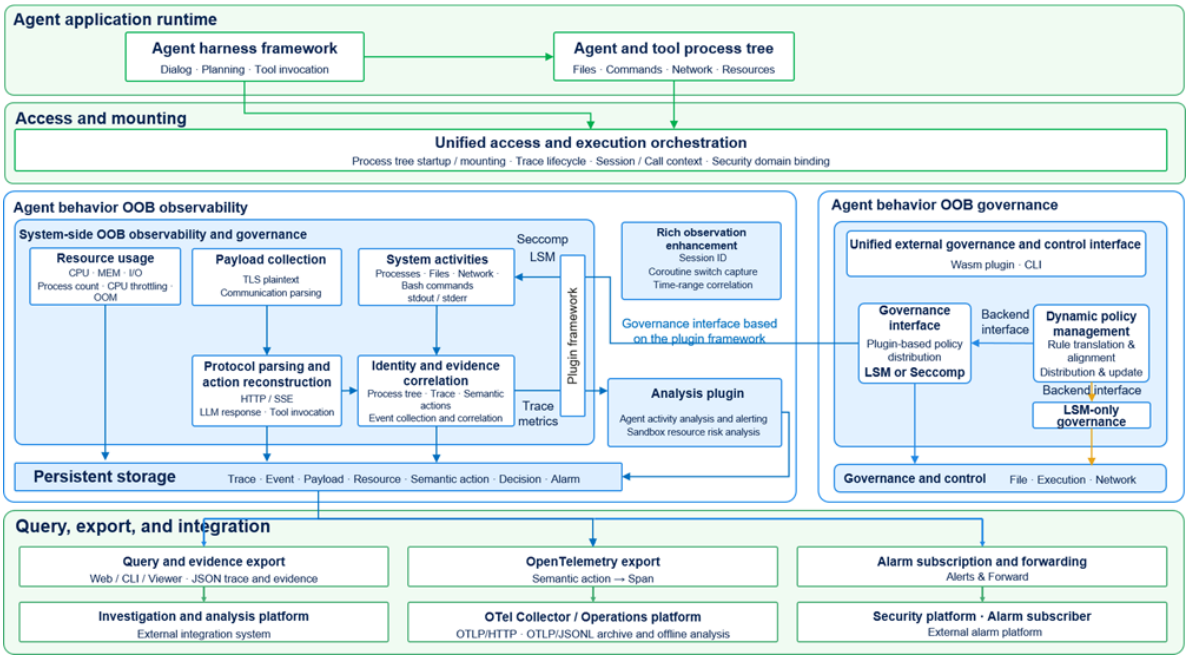
  When model weights are stored on local SSDs, the performance bottleneck often arises from low bandwidth utilization. ModelFS addresses this issue with an I/O template-based mechanism. During the first run, it traces the I/O loading behavior of each inference service.

In subsequent inference startups, it accurately predicts future I/O behavior and fully leverages SSD bandwidth and XPU affinity to accelerate prefetching and eviction.

Agent Observability and Governance

Agents are no longer limited to single-round question answering or code completion. Instead, they have evolved into complex systems with capabilities such as multi-turn reasoning, tool calling, file read/write, script execution, external system access, and cross-agent collaboration. In other words, compared to conventional workflows, autonomous agents exhibit a higher degree of freedom, introducing behavioral uncertainties as well as security and reliability risks. Furthermore, as agents are increasingly deployed in production environments, enterprise priorities are shifting. Historically, system stability was gauged by metrics such as service availability, interface latency, and error rates. Today, evaluating agent trustworthiness requires deep visibility into runtime behavior: what actions were executed, the underlying rationale, the resulting impact, end-to-end auditability, and the capability to govern and control anomalous behavior.

openEuler offers a comprehensive set of capabilities for agent observability and runtime governance, including AcTrail—a service and plugin framework leveraging out-of-band (OOB) system observation—and AgentCensor, which extends observability and provides advanced security protection.



Feature Description

- OOB observation with AcTrail

  AcTrail is a service and plugin framework for OOB agent observability and governance. Through non-intrusive system-side observation and cross-layer behavior correlation, it unifies system activities, model interactions, and sensitive access handling into traceable agent semantic behavior trajectories. Furthermore, it provides flexible foundational governance capabilities to ensure agent runtime security, stability, reliability, and continuous optimization.

  Specifically, for multi-framework agent runtime behavior, AcTrail automatically detects mount points and employs user-space and kernel-space probes to non-intrusively capture user-space protocol payloads (e.g., TLS traffic), kernel-space events (e.g., processes, files,

network activity, and Bash commands), and resource access metrics (e.g., CPU and OOM events). This ensures comprehensive, bypass-resistant coverage of critical agent system activities. Furthermore, AcTrail correlates derived impacts of tool actions beyond the scope of traditional observability, enabling the identification of operations beneath the agent tool declaration layer. Additionally, it provides full observability into the detection and reporting processes of agent harness framework anomalies that elude traditional observability.

It also reconstructs the high-level action execution view based on observed underlying events (such as request fragments, fork calls, and read/write operations), correlating discrete events with specific agent actions to reestablish a comprehensive system-side behavior profile. In addition, it links LLM responses, agent harness activities, agent invocations, and system environment impacts to construct a fine-grained agent behavior impact trace. Moreover, it provides embedded, cross-framework, and programmable control points to support agent security analysis and enforcement, reliability enhancement, and performance optimization.

- Rich-observability AgentCensor

  An agent task often spans multiple conversational turns and tool invocations, generating fragmented execution and resource access records. Standalone events or logs fail to reveal session affiliation, operational dependencies, or the precise invocation stage where anomalies occur. Therefore, a unified execution context must be established atop event records.

  Centered on session-level event correlation, rich observability leverages propagatable session IDs provided by the runtime framework to consolidate model interactions, tool execution, and resource access records into a shared context. By organizing calls, operation targets, and execution results around sessions, it constructs a correlated view spanning from application behavior to system operations, offering full visibility into the task execution lifecycle and associated resource access.

  The correlation mechanism uses session IDs to determine event attribution, establishing cross-layer linkages by combining invocation IDs, process relationships, and event timestamps. For events with framework-provided start and stop information enabling unique matching, it further correlates them with specific invocations. Event records, correlation relationships, and evidence references are stored in an SQLite database equipped with session-time and invocation-time composite indexes, supporting the tracing of relevant system behavior and correlation evidence from application invocations.

  On this basis, database read-only views enable session-based queries of event sequences and correlated behavior, tracking commands, file access, and network activity associated with linked invocations, as well as performing reverse lookups from anomalies to their parent sessions and invocation contexts. Furthermore, correlation results can be exported in JSON format for execution analysis, anomaly localization, and security auditing, thereby eliminating cross-log searching and manual correlation efforts.

- Dynamic control and governance

  An agent's execution paths and resource requirements cannot be fully determined before a task begins, while tool invocations may trigger file modifications, program execution, or external connections. Overly permissive static privileges amplify security risks, whereas overly strict restrictions impede normal task execution. Therefore, security constraints must be enforced at runtime, with support for adjusting access policies based on task demands.

  Designed for kernel environments supporting eBPF Linux Security Module (LSM), the dynamic control and governance capability enforces security policies along resource access paths. By attaching eBPF programs to LSM hooks—such as file open and permission checks, program loading, and network connections—the kernel synchronously queries rules to render allow or deny decisions, returning permission errors for non-compliant operations to achieve runtime security interception.

The control layer binds the target process tree to a governance domain, using BPF maps to maintain mapping relationships among processes, governance domains, and policies, alongside file, program, and network access rules. Child processes inherit their parent's governance domain upon creation, and subsequent operations are evaluated against the active domain policy to constrain accessible file scopes, executable programs, and connection target addresses and ports.

Policy updates employ a dual-rule-region mechanism: new policies are validated and written to the standby rule region before switching the active rule region, eliminating the need to restart governed processes. The new policy takes effect for subsequent operations triggering security checks, with support for rolling back to historical policies on demand. Additionally, processes, governance domains, evaluation results, and policy versions are logged based on configurations to provide evidence for investigating interception causes and auditing policies. Furthermore, it provides users with programmable interfaces for analysis and control based on observed underlying facts and reconstructed actions, allowing users to integrate plugins to implement extensive control logic and safeguard agent runtime security on demand.

Application Scenarios

- Traceable, fine-grained evidence chains and post-execution auditing

  Model responses, tool invocations, and system calls (such as process launching, file access, network access, and command execution) are assembled into complete invocation chains and action execution trees. This enables direct retroactive inspection of execution records without log aggregation, satisfying compliance auditing and incident tracing requirements.

- Human-in-the-loop review checkpoints

  When manual intervention is required, execution is suspended pending a decision: approval resumes execution, whereas rejection triggers automatic rollback. This prevents agents from timing out autonomously or performing irreversible operations.

- Defense against agent out-of-control scenarios and supply chain attacks

  In scenarios where an agent executes destructive commands (such as database deletion), is injected with malicious inputs (such as credential theft), or has poisoned dependencies installed, this capability enforces pre-execution synchronous evaluation and write isolation to confine potential damage within a rollback-supported scope, bridging the security gap of "prompt + sandbox" defenses at the internal agent tool invocation layer.

Context Reduction via Data Referencing

Feature Description

During long-running agent execution, message histories continuously accumulate voluminous tool outputs (such as file read results, command execution outputs, and scraped web contents). The majority of these contents remain unused in subsequent turns, yet they consume context window capacity and increase token costs. Simple Compression Context Storage (SCCS) addresses this issue using the following methods:

- **Compression and replacement (offloading original data)**: For successfully returned tool outputs exceeding 3,000 characters, the original text is offloaded from the context and replaced with a lightweight reference representation **[REF_ID: xxx] (Summary: ...)**. This comprises a reference identifier accompanied by a structured summary containing key structural information, such as code line numbers and function signatures, while the original text is temporarily cached in the backend storage.
- **On-demand retrieval (restoring original data)**: When the model requires a specific block of compressed original text during subsequent task execution, it invokes

fetch_original_data(ref_ids=[...]). SCCS returns the complete original text based on the specified **ref_id**, enabling data restoration at the reference granularity.

This workflow establishes an "**offloading → summary → on-demand restoration**" closed loop, functioning as an "external memory and paging mechanism" for the agent context: the indexes (summaries) remain resident in the context, while the data payloads are paged in on demand.

The effectiveness of on-demand loading relies on the model's awareness of the reference mechanism and its usage. SCCS ensures this through three channels:

Channel	Content	Function
Tool definition	Injects the **fetch_original_data** tool (name/description/parameters) into the model toolset.	Informs the model that data can be retrieved.
Inline compression prompt	Embeds a system notice (**[SYSTEM NOTE - REF_ID HANDLING]**) directly within each compressed result.	Instantly notifies the model how to retrieve the data at the point of compression.
Context-aware prompt injection	Appends a high-priority decision directive (**REF_ID_INSTRUCTION**) to the prompt in each turn.	Continuously guides the model on when and how to retrieve data.

- **Compression triggering policy**: Not all tool outputs warrant compression. SCCS triggers compression only when all of the following conditions are met: successful execution (error results are never compressed and are passed through directly to prevent the model from missing critical error information), presence of text content, volume exceeding the threshold (> 3,000 characters), and non-retrieved status (to prevent a retrieve-then-recompress loop). Compression hotspots are concentrated in three scenarios: reading large files (**read**), large-output commands (**exec**, such as **grep**/**git log**), and web scraping (**web_fetch**).
- **Zero-intrusive integration**: SCCS is integrated via the native plugin mechanism of the host agent framework (such as OpenClaw activation via **agent --local** and delivery of the retrieval tool via **tools.alsoAllow**), requiring no modifications to the host code.
- **Session/Process isolation**: Each task session utilizes an independent state directory and session key, ensuring that data retrieval operations do not interfere with one another.
- **Pluggable storage backends**: Backends such as process memory (**memory**, non-persistent) are supported, allowing the lifecycle of the original text and deployment forms to be selected on demand.

Application Scenarios

- **Agent tasks with long conversations and multi-tool invocations**: Extends the achievable task depth of agents operating within limited context windows.
- **Deployment environments with limited context windows**: Reduces input-side overhead, allowing the system to accommodate longer task histories or more complex task orchestrations.
- **Automated code fixing/software engineering tasks**: Validates that summaries provide sufficient information for issue localization, and original texts can be retrieved when necessary.
- **Batch evaluation and production-grade agent services**: Controls context scale, providing a stable foundation for scalable and reproducible evaluations and services.

- **RAG/Multi-source information consumption applications**: Well-suited for read-heavy, write-light scenarios such as customer support Q&A, report generation, and in-depth research.

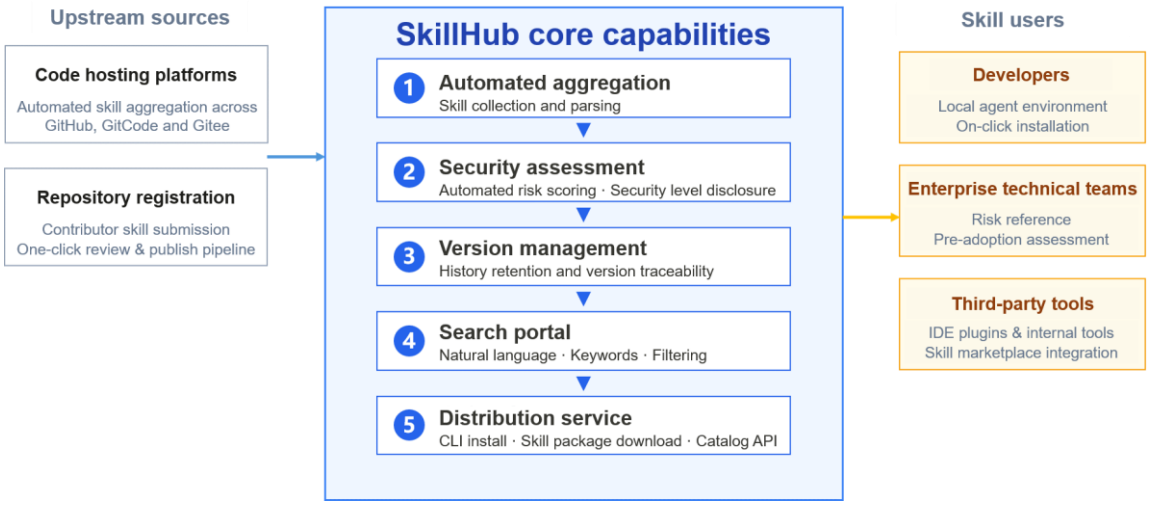
SkillHub

Feature Description

SkillHub is an AI agent skills retrieval and distribution platform developed and operated by the openEuler community. By aggregating, automatically assessing, and publicly distributing AI skills scattered across various code hosting platforms, SkillHub enables developers to confidently discover, evaluate, use, and contribute AI capabilities.

The platform provides the following capabilities:

- **Skill discovery**: Allows users to search for matching AI skills using natural language or keywords, with multi-dimensional filtering by domain, contributor source, and security level for rapid skill location.
- **Security assessment and information transparency**: Subjects every published skill to automated security evaluations, publicly presenting quantitative risk scores and intuitive security levels. This provides users with objective risk information to make informed decisions tailored to their specific deployment scenarios.
- **Version traceability**: Retains complete version histories, enabling users to track version evolution and select specific versions for download and deployment.
- **Convenient acquisition**: Enables one-command installation into local agent environments upon finding suitable skills, as well as direct downloading of skill packages for self-managed integration.
- **Open contribution**: Allows contributors to publish skills simply by registering a public repository containing a skill specification file in the community contribution repository; the platform automatically handles subsequent workflows once approved.
- **Ecosystem convergence**: Continuously and automatically collects community and third-party skills to build an ever-growing skill repository, reflecting popularity of skills via usage metrics.



Application Scenarios

- **Skill discovery and integration**: Enables agent developers to retrieve matching skills simply by describing required functionalities, review security levels and usage documentation, and install them with a single command, eliminating the need to search through individual repositories and perform manual integration.
- **Pre-adoption risk assessment**: Allows enterprises to filter skills by security level and inspect public risk metrics and evaluation reports to assess suitability for their specific business scenarios, shifting risk identification forward to the selection stage.
- **Publication of proprietary skills**: Enables community SIGs, enterprises, organizations, and individual developers to publish reusable skills through a one-time registration; subsequent updates are handled automatically by the platform without requiring ongoing platform maintenance.
- **Skill ecosystem consumption by third-party tools**: Allows IDE plugins and internal tool marketplaces to integrate the platform's API and present the skill catalog within their products, enabling users to access AI skills directly from familiar tools.
- **Ecosystem cold start and continuous growth**: Automatically aggregates existing community skills to form initial content, while leveraging low-barrier community contributions and usage metrics to drive sustained ecosystem expansion.

5 Innovations for Server Scenarios

oeAware Scenario-based Tuning Skill Framework

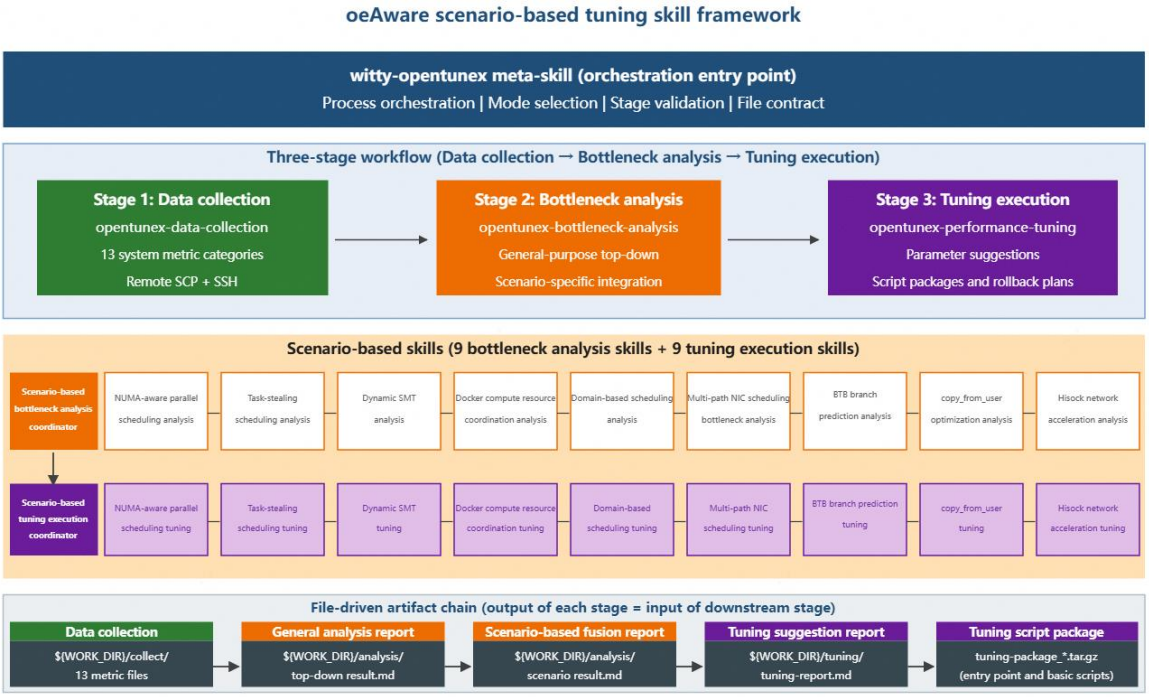
Feature Description

The oeAware scenario-based tuning skill framework provides intelligent capabilities for OS-level performance tuning. It consists of four core skills that work together: witty-opentunex (orchestration meta-skill), opentunex-data-collection (data collection), opentunex-scenario-bottleneck (scenario-specific bottleneck analysis), and opentunex-scenario-tuning (scenario-specific tuning execution). The framework transforms traditionally expertise-driven bottleneck identification and parameter tuning into an automated workflow driven by large language model (LLM) inference, with support for remote execution and phased rollout and rollback.

GLM-4.7, MiniMax M2.7, or a model with equivalent or higher capabilities is recommended. The model should support a context length of more than 200,000 tokens.

The framework adopts a four-layer architecture. The orchestration meta-skill layer manages workflow orchestration and mode selection. The three-stage workflow layer covers data collection, bottleneck analysis, and tuning execution. The scenario-specific capability layer provides nine analysis sub-skills and nine tuning sub-skills. The file-driven contract and artifact layer connects upstream and downstream stages through standardized path conventions.

Functional architecture of the oeAware scenario-based tuning skill framework



Application Scenarios

The oeAware scenario-based tuning skill framework supports OS-level performance tuning for databases, caches, message queues, big data, and other workloads. Users can integrate the skill package into an agent and invoke witty-opentunex to initiate a session. The agent then automatically performs data collection, analyzes OS-level bottlenecks, and generates scenario-specific tuning solutions, enabling rapid bottleneck identification and deployment of reversible tuning scripts.

Mapping between scenarios and workloads

Scenario	Typical Workload	Identifiable Bottleneck	Tuning Capability
NUMA-aware parallel scheduling	In-memory databases, HPC workloads, and LLM inference on multi-NUMA servers	High cross-NUMA access rate, constrained memory bandwidth, and high remote-memory hit rate	**PARAL** (AArch64)
Task-stealing scheduling	Multicore general-purpose workloads with high CPU utilization	Severe CPU load imbalance, high **%steal/sys** values, and underutilized idle cores	**STEAL** (AArch64)
Dynamic SMT	Workloads with low CPU utilization and SMT interference	Low CPU utilization and single-thread latency degradation caused by sibling threads	**KEEP_ON_CORE**

Scenario	Typical Workload	Identifiable Bottleneck	Tuning Capability
Docker compute resource coordination (burst)	Containerized databases, microservices, and online services	Task loss due to container CPU throttling despite available host CPU cores	**sched_soft_runtime_ratio** + **cpu.soft_quota**
Domain-based scheduling	Multi-NUMA environments with multiple containers/processes and small CPU quotas	Cross-NUMA scheduling jitter and coarse-grained scheduling domains	**SOFT_DOMAIN** (AArch64)
Multi-path NIC scheduling	Network-intensive workloads running across multiple NICs and NUMA nodes	Poor cross-NUMA interrupt affinity and network jitter	**oenetcls**/**venetcls** + **ntuple**
BTB branch prediction	Thread-intensive workloads such as Redis/MySQL on Kunpeng 920 with frequent context switching of critical threads	BTB pollution across threads and reduced IPC of critical threads	Disable TidCMP in BIOS
copy_from_user optimization	Workloads with heavy large-block read/write traffic on ARM64 HiSilicon CPUs	High CPU time spent in **copy_from_user**/**copy_to_user** as observed by perf	**ldp**/**ldtp** kernel patches
Hisock network acceleration	Short-connection and high-PPS workloads, such as gateways, proxies, and NFV services	Significant **nf_hook** hotspots and high L2/L3 netfilter overhead	eBPF acceleration

- **NUMA-aware parallel scheduling**
  - **Applicable workloads**: Servers with multiple NUMA nodes

    When in-memory databases (such as Redis), HPC applications, or LLM inference services are deployed on multi-socket servers, conventional affinity policies may cause processes to be concentrated on a small number of NUMA nodes in the absence of NUMA-aware parallel scheduling. This can result in a high cross-NUMA access rate, constrained memory bandwidth, and a high remote-memory hit rate, ultimately reducing throughput and increasing tail latency. The **opentunex-numa-sched-analysis** sub-skill analyzes the NUMA topology and cross-node access metrics collected from the workload. It determines whether the workload exhibits NUMA-unfriendly access patterns. If a bottleneck is identified and the system supports the **PARAL** kernel feature, the tuning stage generates recommendations to enable PARAL and configure **sched_util_low_pct**. The **numa_sched_tune.sh** tuning script is then used to apply the configuration.

  - **Expected benefits**: Reduced cross-NUMA access latency, improved memory bandwidth utilization, and a 5%–20% increase in throughput for multi-NUMA workloads.

- **Task-stealing scheduling**
  - **Applicable workloads**: Multicore workloads with high CPU utilization

    For multicore general-purpose workloads with high CPU utilization, such as compilation, parallel computing, and video transcoding, significant CPU load imbalance across physical cores may result in some cores being fully utilized while others remain idle. This can lead to high **%steal**/**sys** values and limit overall throughput. The **opentunex-stealtask-analysis** sub-skill analyzes CPU load balance and the **SCHED_STEAL** kernel configuration. If a bottleneck is identified and the kernel supports the **STEAL** feature, **opentunex-stealtask-tuning** generates specific recommendations for enabling the **STEAL** feature.

  - **Expected benefits**: Improved CPU load balance, a 10%–20% increase in overall CPU utilization, and reduced task scheduling latency.

- **Dynamic SMT**
  - **Applicable workloads**: Workloads with low CPU utilization and SMT interference

    On servers with SMT enabled, when overall CPU utilization is low, conventional schedulers may schedule two threads from the same task to the two logical CPUs of the same physical core. This can cause contention for shared SMT resources and increase single-thread latency due to interference from sibling threads. The **opentunex-dynamic-smt-analysis** sub-skill identifies the CPU utilization level and SMT topology and determines whether the workload is suitable for the **KEEP_ON_CORE** kernel feature. For applicable scenarios, **opentunex-dynamic-smt-tuning** generates recommendations for enabling **KEEP_ON_CORE**, allowing low-load tasks to exclusively occupy physical cores.

  - **Expected benefits**: Improved single-thread latency stability under low-load conditions and elimination of SMT interference.

- **Docker compute resource coordination (burst)**
  - **Applicable workloads**: Containerized workloads

    In Kubernetes and containerized environments, workloads typically use the cgroup **cpu.cfs_quota_us** parameter to limit CPU usage for individual containers. During workload bursts, CPU throttling may cause tasks to be dropped or queued while idle CPU resources remain available on the host. The **opentunex-docker-coordination-burst-analysis** sub-skill identifies container CPU throttling and available host CPU resources, and determines that the current tuning approach is applicable to environments supporting the **sched_soft_runtime_ratio** kernel feature. During the tuning stage, **opentunex-docker-coordination-burst-tuning** generates specific recommendations for configuring **sched_soft_runtime_ratio** and **cpu.soft_quota** (for example, **ratio=20**), allowing containers to use idle host CPU resources during bursts.

  - **Expected benefits**: Improved container CPU utilization, reduced tail latency, and a 10%–30% increase in resource utilization.

- **Domain-based scheduling**
  - **Applicable workloads**: Multi-NUMA environments with multiple containers/processes

    In deployments with multiple instances and small CPU quotas, such as multi-replica Kubernetes deployments, coarse-grained scheduling domains may cause processes to frequently migrate between NUMA nodes, resulting in cross-NUMA scheduling jitter. The **opentunex-soft-domain-analysis** sub-skill analyzes the NUMA topology and container and process deployment to determine whether the **SOFT_DOMAIN** scheduling feature is

applicable. During the tuning stage, it generates a script for enabling domain-based scheduling.

  - **Expected benefits**: Reduced cross-NUMA migration jitter and improved scheduling latency stability.

- **Multi-path NIC scheduling**
  - **Applicable workloads**: Network-intensive workloads running across multiple NICs and NUMA nodes

    In multi-NIC, multi-NUMA topologies, improper interrupt affinity configuration may result in cross-NUMA interrupt processing, network jitter, and uneven NIC utilization, with one NIC reaching capacity while others remain underutilized. The **opentunex-multi-net-path-analysis** sub-skill analyzes NIC, NUMA, and IRQ affinity configurations to determine whether multi-path tuning is applicable. For applicable scenarios, **opentunex-multi-net-path-tuning** generates recommendations to enable the **oenetcls** and **venetcls** kernel modules for interrupt processing and configure **ntuple**.

  - **Expected benefits**: Optimized cross-NUMA network interrupt affinity and improved network throughput and latency stability.
- **BTB branch prediction**
  - **Applicable workloads**: Workloads with critical threads on Kunpeng 920

    On Kunpeng 920 and other newer Arm processors, BTB resources are partitioned by thread to reduce cross-thread branch prediction pollution. However, for workloads with a large number of threads, such as Redis and MySQL, frequent switching of critical threads may cause thread-level partitioning to limit the BTB hit rate of critical threads, resulting in reduced IPC. The **opentunex-btb-analysis** sub-skill determines whether tuning is applicable based on the CPU model and the characteristics of critical processes. During the tuning stage, **opentunex-btb-tuning** generates BIOS configuration recommendations to disable **TidCMP**, which enables BTB resources to be shared across threads.

  - **Expected benefits**: Improved BTB hit rate for critical threads, increased single-thread IPC, and reduced tail latency.
- **copy_from_user optimization**
  - **Applicable workloads**: ARM64 workloads with large-block read/write hotspots

    On ARM64 HiSilicon CPUs, workloads with large-block read/write hotspots, such as network I/O and encryption/decryption, may encounter significant performance bottlenecks because conventional **copy_from_user**/**copy_to_user** implementations do not fully utilize the ARM64 double-word load instructions (**ldp**/**ldtp**). The **opentunex-copy-user-analysis** sub-skill identifies applicable scenarios by analyzing perf hotspots to determine whether **copy_from_user**/**copy_to_user** account for a significant proportion of CPU time and whether the CPU model meets the requirements. During the tuning stage, **opentunex-copy-user-tuning** generates recommendations to apply the **ldp**/**ldtp** double-word load optimization kernel patches and rebuild the kernel.

  - **Expected benefits**: Increased user-to-kernel and kernel-to-user copy throughput and reduced CPU overhead of hotspot functions.
- **Hisock network acceleration**
  - **Applicable workloads**: Gateways and proxy workloads with short connections and high PPS

    In gateway, proxy, and NFV workloads with short connections and high PPS, **nf_hook** is a common hotspot, and the overhead of the L2/L3 netfilter framework can significantly affect PPS performance. The **opentunex-hisock-analysis** sub-skill identifies applicable

scenarios when **nf_hook** is identified as a significant hotspot in perf analysis. During the tuning stage, **opentunex-hisock-tuning** generates an eBPF-based hisock acceleration solution, including specific steps for compiling and loading the eBPF programs, allowing packets to bypass parts of the netfilter path.

- **Expected benefits**: Increased PPS for short-connection workloads, reduced L2/L3 netfilter overhead, and improved network throughput and latency.

6 Developer Toolchain

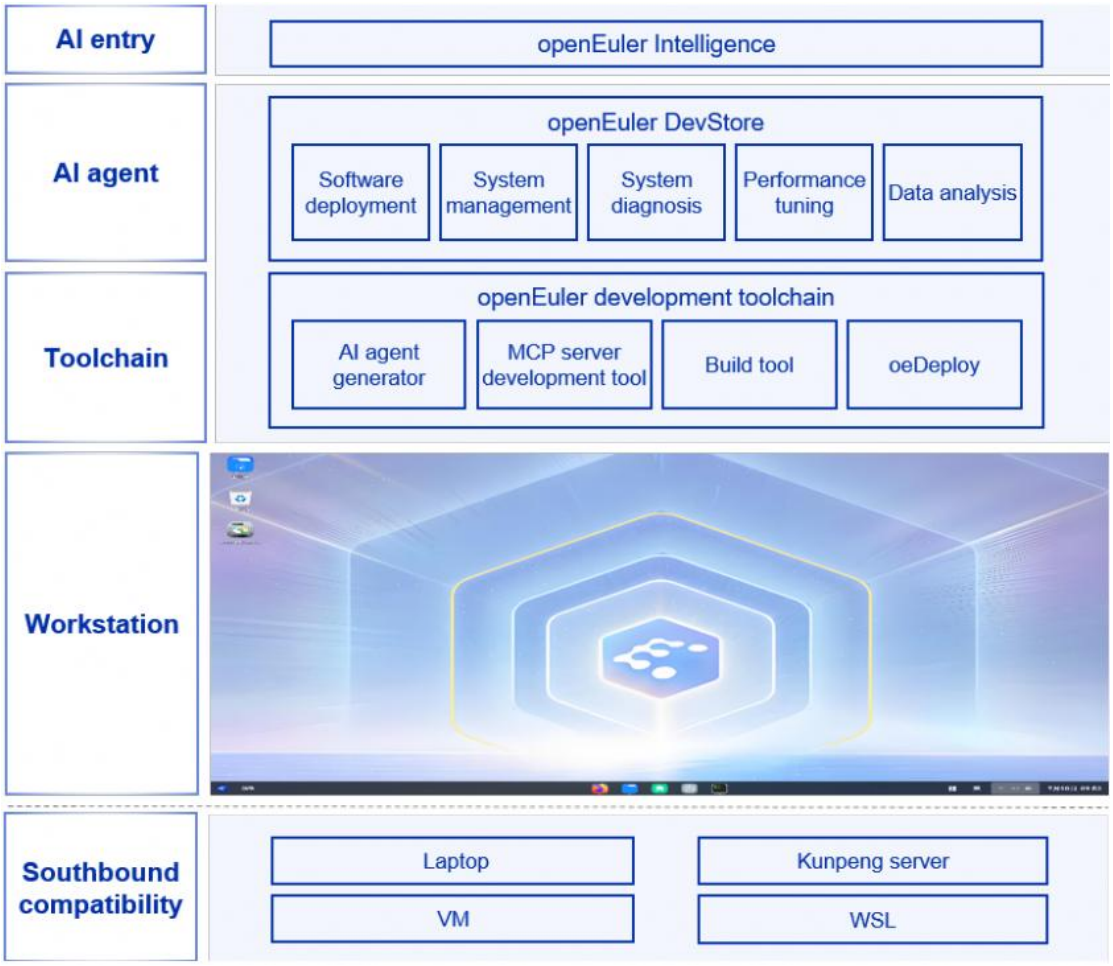
DevStation

DevStation is an intelligent developer workstation based on openEuler, designed for tech enthusiasts and innovators. It provides a ready-to-use, efficient, and secure development environment that streamlines the entire workflow from deployment and coding to compilation, build, and release.

By combining one-click runtime environment setup with a full-stack development toolchain, DevStation streamlines the development process from system boot to code delivery. Developers can get started without complex installation or configuration. Its new MCP-powered AI engine enables rapid access to community toolchains, improving efficiency from infrastructure setup to application development.

Feature Description

- **Developer-friendly integrated environment**: The distribution comes preinstalled with a wide range of development tools and IDEs, including VSCodium-based IDEs. It supports multiple programming languages and meets the needs of frontend, backend, and full-stack development.
- **Native community tool ecosystem**: The distribution includes oeDeploy for one-click deployment, epkg as an extended package manager, DevKit, and openEuler Intelligence, providing end-to-end support from environment setup to code delivery.
  - **oeDevPlugins and oeGitExt for community development**: Designed for openEuler community developers, the oeDevPlugins VSCodium extension provides a visual dashboard for managing issues and pull requests (PRs). Together with the oeGitExt command-line tool, it enables developers to quickly clone community repositories, submit PRs, and track community task status in real time.
  - **openEuler Intelligence**: The intelligent assistant supports natural-language generation of code snippets, one-click API documentation generation, and explanations of Linux commands. Integrated with the epkg package manager, it also streamlines the installation and deployment of applications, including AI agents, in openEuler environments.



- **Visual programming environment**: The distribution integrates visual programming tools to make programming more accessible to beginners while providing advanced developers with powerful visual programming capabilities. It also comes preinstalled with productivity tools such as Thunderbird.
- **MCP-powered intelligent application ecosystem**: DevStation deeply integrates the Model Context Protocol (MCP) framework to build a comprehensive ecosystem of intelligent tools. It comes preinstalled with an MCP toolchain that supports key MCP servers, including oeGitExt and rpm-builder, enabling capabilities such as community task management and RPM packaging. By exposing traditional development tools, such as Git and RPM builders, through MCP, DevStation enables developers to interact with these tools using natural language.
- **System deployment and compatibility enhancements**: The distribution offers broad hardware support, with improved compatibility with mainstream laptop and PC hardware, particularly touchpads, Wi-Fi, and Bluetooth. The kernel build scripts (kernel-extra-modules) have been restructured to ensure reliable bare-metal deployment. DevStation supports multiple deployment options, including LiveCD (boot and run without installation), bare-metal installation, and virtual machine deployment.
- **New installer, heolleo**: heolleo is a modern installation tool designed specifically for DevStation to streamline the installation process. Its modular architecture makes it easy to extend support for different hardware architectures, such as x86 and Arm, as well as different file systems and bootloaders, including GRUB. It also adopts an innovative SquashFS-based installation approach to significantly reduce the size of standard images, reducing download costs for developers. For users who prioritize stability and speed or need to deploy the

system in offline or restricted environments, heolleo supports installation from a local ISO image. This approach reuses existing system image files to deliver a fast, reliable, and fully offline installation experience.

Application Scenarios

- **Multi-language development environment**: Designed for developers working on projects in multiple programming languages, such as Python, JavaScript, Java, and C++, DevStation comes preinstalled with a range of compilers, interpreters, and build tools, eliminating the need for manual environment setup.
- **Rapid installation and deployment**: DevStation integrates oeDeploy to enable the deployment of distributed software platforms such as Kubeflow and Kubernetes within minutes, significantly reducing deployment time. oeDeploy also provides a unified plugin framework and modular deployment capabilities. By following simple development guidelines, developers can quickly publish custom installation and deployment plugins, helping more users address installation and deployment challenges.
- **Hardware compatibility testing and development**: DevStation supports mainstream laptops and servers, with compatibility validated on supported hardware configurations. It also supports deployment on personal PCs for driver testing.
- **Improved developer productivity**: The MCP rpm-builder toolchain streamlines MCP adoption by automating the packaging of MCP servers into RPM packages and publishing them to the openEuler community. It is designed to ensure reliable one-click installation of supported MCP servers. This lays the foundation for a comprehensive repository of MCP-powered applications covering deployment, testing, and performance tuning. Supported workflows include checking assigned community issues, submitting code changes through PRs, and automatically building and validating changes through CI/CD.
- **Standardized environment for AI and agent development**: DevStation provides a standardized installation and deployment environment that meets Agent toolchain requirements, with support for Node.js runtime version management, observability, and other essential infrastructure capabilities.
- **Cross-distribution software support**: Developers are often constrained by the software ecosystem of the host OS, particularly when required application dependencies are unavailable in openEuler. With epkg, developers can create isolated environments associated with openEuler, Debian, or Alpine channels to access software from different distributions without modifying the host system. This simplifies the installation and deployment of applications, including agents, on openEuler.

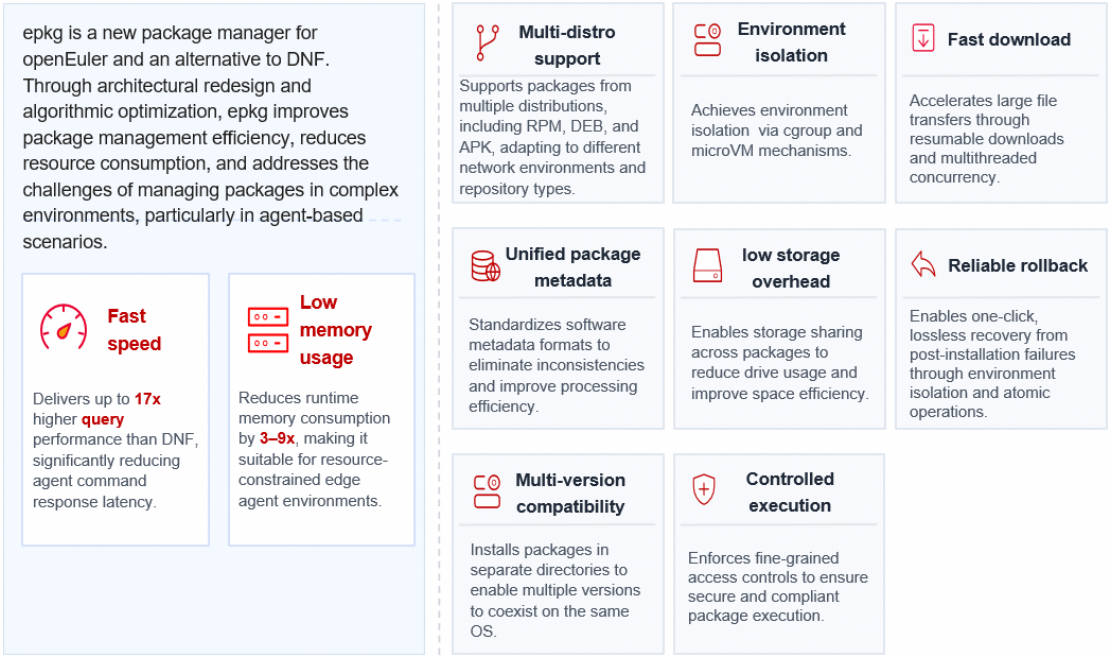
epkg

epkg is a lightweight, cross-platform package manager for Linux, macOS, and Windows. It allows users to create isolated environments and install packages from ecosystems such as RPM, DEB, Alpine, Arch, Conda, Homebrew, and MSYS2 without root privileges. Each environment is bound to a specific channel, such as a channel for a particular version of Debian, Fedora, or Alpine. By registering multiple environments and adding their binary directories to **PATH**, users can run software from different ecosystems within the same shell.

Feature Description

epkg uses a user-space installation model that allows packages, environments, and caches to be stored in user directories. Users can install, upgrade, and remove software and manage environments without administrator privileges, minimizing changes to the host system. Through a unified channel mechanism, epkg integrates with multiple software ecosystems. It enables users to access packages from distributions such as openEuler, Fedora, CentOS, Debian, Ubuntu, Alpine, and Arch Linux on the same host system. It also supports ecosystems such as Conda,

Homebrew, and MSYS2. This approach decouples project-specific software environments from the host OS, reducing reliance on the host system's native package manager and allowing projects to use dependencies from different software ecosystems.



Environment isolation is one of epkg's core capabilities. Each environment can be bound to a specific channel and software version, with packages managed independently across environments. Environments can be activated, registered, unregistered, and combined as needed. Users can maintain multiple independent software environments simultaneously. For example, they can create separate environments with Python 3.10, Python 3.11, or Python 3.12 for different projects without repeatedly changing the Python version on the host system. Different versions of compilers, runtimes, and system tools can coexist in the same way. Environment export and import capabilities also enable development environments to be replicated and shared.

epkg further unifies the management of OS-level and language-level dependencies within a single environment. Conventional development environments often require separate tools, such as apt, dnf, apk, pacman, Conda, and pip, to install dependencies at different levels, leaving dependency management fragmented across multiple package management systems. epkg allows OS packages and language runtimes to coexist in the same environment, enabling projects to bring together compilers, system libraries, Python runtimes, and other development tools. For example, an AI agent project may require specific versions of Python and Git, build tools, system libraries, and other runtime components. Developers can manage all these dependencies in one place, reducing tool switching during environment setup and helping prevent dependency conflicts.

For dependency resolution and package management, epkg uses a SAT-based dependency solver to handle complex software dependencies and provides standard package operations, including **install**, **update**, **upgrade**, **remove**, **list**, **search**, and **info**. It also maintains a transaction history and supports **restore**, allowing environments to be restored to a previous state after software updates or unexpected issues. epkg uses file-level deduplication for software storage and combines parallel, chunked downloads with mirror support to reduce storage and network overhead associated with redundant downloads. The project also provides a statically

linked musl executable of approximately 14 MB and supports BusyBox-style applets, making epkg suitable for resource-constrained environments.

Beyond conventional package management, epkg provides extended capabilities ranging from software installation to runtime environment isolation. The **epkg run** command supports multiple isolation modes, including environment isolation based on user namespaces and bind mounts, file system isolation based on **chroot**, and VM-based isolation. This makes epkg more than a software installation tool. It can also serve as a foundational component for building lightweight development environments and isolated runtime environments. In scenarios such as containerized workloads, embedded systems, and local AI development, epkg provides software acquisition, dependency management, environment organization, and isolated execution with low system overhead.

Application Scenarios

- **Cross-distribution software access**: Developers are often limited by the software ecosystem of the host OS. For example, a server may run openEuler while one project depends on packages from the Debian ecosystem and another tool is primarily available through Alpine or Arch. Conventional approaches often require changing the host environment, deploying additional containers, or manually handling package formats and dependencies. With epkg, developers can create separate environments bound to openEuler, Debian, or Alpine channels to access software from different distributions without modifying the host system. By registering these environments, they can add the required tools to the current shell's PATH and use software from multiple distributions side by side.
- **Multi-version coexistence and environment isolation**: AI development is particularly prone to dependency conflicts. Different agent projects, for example, may require Python 3.10, 3.11, or 3.12, along with different versions of Node.js, compilers, or system libraries. Changing the system Python version or repeatedly switching versions through a global package manager can disrupt existing projects. With epkg, developers can create a separate environment for each agent project and install the required Python version and other dependencies in that environment. They can then activate the appropriate environment when working on a project. Packages are isolated across environments, preventing changes in one environment from directly affecting software in others. This provides a stable dependency foundation for parallel development across multiple agents and projects.
- **Unified management of OS-level and language-level dependencies**: An AI application may depend on a Python runtime, third-party Python libraries, Git, cURL, build tools, shared system libraries, and other command-line utilities. Conventional approaches often require a combination of system package managers, Conda, and pip, with dependency configurations maintained separately. epkg brings OS packages and language runtimes into a single environment and uses a unified environment description and export mechanism to organize project dependencies, making development environments easier to reproduce on other machines. For AI agent development, where development environments are frequently created and removed, this approach to managing dependencies across multiple layers in a single environment reduces environment setup and maintenance costs.
- **Development and computing without root privileges**: Users often lack administrator privileges on servers or shared workstations but still need to install specific software versions. For example, users on enterprise servers, research clusters, and shared GPU workstations may need to install Python, Git, compilers, or AI development tools independently. epkg allows users to install software in their own directories and environments without modifying system-level software configurations, making it suitable for development environments with restricted privileges. The official openEuler documentation also describes how to use epkg as a non-root user.
- **AI agent development and local execution**: epkg enables developers to create dedicated environments for individual agents or projects and use **epkg run** with isolation options to

execute commands in isolation, keeping each agent's software dependencies relatively independent of the host system. For example, one agent may require Python 3.11 and specific system tools, while another may require Python 3.12 and different runtime versions. Developers can configure these environments independently and select the appropriate one for each task. Combined with file system or VM-based isolation, epkg can serve as an environment management foundation for lightweight agent sandboxes, helping manage software dependencies and control the execution environment for agent tool calls.

- **Containers and embedded systems**: Conventional distribution package managers often come with substantial system components and runtime dependencies. When building minimal containers or system images for resource-constrained environments, the package manager and its dependencies can add significant overhead. epkg provides a static executable and unified package management across ecosystems such as RPM, DEB, and APK, making it suitable as a lightweight package manager for containers. File-level deduplication helps reduce storage requirements, while epkg's small footprint limits the overhead introduced by the package manager itself. The project's stated design goals include minimizing the additional storage overhead of package management tools in RPM- and DEB-based containers.

Overall, epkg goes beyond serving as an alternative to conventional package managers such as APT and DNF. It provides cross-platform software environment management built around unified software sources, environment isolation, unified dependency management, and a lightweight design. By bringing software from multiple distributions, multiple software versions, and both OS-level and language-level dependencies into a unified environment model, epkg extends these capabilities to use cases such as containers, AI development, and agent isolation. This enables developers to organize software environments independently of the host OS.

7 Copyright Statement

All materials or contents contained in this document are protected by the copyright law, and all copyrights are owned by openEuler, except for the content cited by other parties. Without a prior written permission of the openEuler community or other parties concerned, no person or organization shall reproduce, distribute, reprint, or publicize any content of this document in any form; link to or transmit the content through hyperlinks; upload the content to other servers using the "method of images"; store the content in information retrieval systems; or use the content for any other commercial purposes. For non-commercial and personal use, the content of the website may be downloaded or printed on the condition that the content is not modified and all rights statements are reserved.

8 Trademarks

All trademarks and logos used and displayed on this document are all owned by the openEuler community, except for trademarks, logos, and trade names that are owned by other parties. Without the written permission of the openEuler community or other parties, any content in this document shall not be deemed as granting the permission or right to use any of the aforementioned trademarks and logos by implication, no objection, or other means. Without prior written consent, no one is allowed to use the name, trademark, or logo of the openEuler community in any form.

9 Appendixes

Appendix 1: Setting Up the Development Environment

Environment Setup	URL
Downloading and installing openEuler	https://www.openeuler.org/en/
Preparing the development environment	https://atomgit.com/openeuler/community/blob/master/en/contributors/prepare-environment.md
Building a software package	https://atomgit.com/openeuler/community/blob/master/en/contributors/package-install.md

Appendix 2: Security Handling Process and Disclosure

Security Issue Disclosure	URL
Security handling process	https://atomgit.com/openeuler/security-committee/blob/master/docs/en/vulnerability-management-process/security-process-en.md
Security disclosure	https://atomgit.com/openeuler/security-committee/blob/master/docs/en/vulnerability-management-process/security-disclosure-en.md
Security strategy overview	https://atomgit.com/openeuler/security-committee/blob/master/docs/en/vulnerability-management-process/security-strategy-overview-en.md